

计算机组织与结构实验指导书

软件部分

信息科学与技术学院
2023.4

目录

实验一 Logisim 使用	3
实验二 数据传输实验	9
实验三 数据运算实验	15
实验四 数据存储实验	23
实验五 利用 DEBUG 调试汇编语言程序段	31
实验六 内存操作数及寻址方法	38
实验七 汇编语言程序的编辑、编译、连接及调试方法	39
实验八 屏幕字符显示程序	41
实验九 循环程序设计（该实验可选做）	43

实验一 Logisim 使用

1. 熟悉环境

学习使用 logisim，熟悉基本功能。

2. 实验内容

下载 [Logisim 软件](#)，启动 Logisim 应用程序。

2.1 基本功能

✧学会使用 toolbar 上的功能。

✧学会增加子电路，并能够将子电路放到 main 电路中或者其他电路中使用

2.2 ToolBar 主要功能

我们将通过创建一个非常简单的电路来感受一下如何放置门和电线。

- 1、首先，单击“AND gate”  按钮。这时鼠标附近会出现一个与门的图标，在主电路图窗口任意位置单击鼠标以放置与门。
- 2、单击“Input Pin”  按钮。在你的与门左侧放置两个输入(input pin)。
- 3、单击“Onput Pin”  按钮。在你的与门右侧放置一个输出(output pin)。这时你的电路图看上去可能如下图所示：

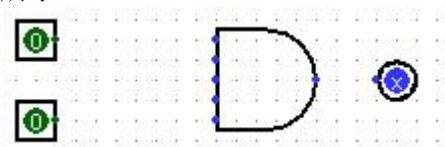


图 1-1 添加元件

- 4、单击“Wire tool”按钮 。单击并拖动它，以便将输入端和与门的左边相连。如果你只画垂直电线和水平电线的话，这一步可以分成几步。首先画一条水平电线，放开鼠标，单击并拖动电线的末端画一条垂直电线。你可以把电线连接到与门左边的任意一条腿上。重复这一过程，把与门的输出和 LED 相连。这时你的电路图看上去可能如下图所示：

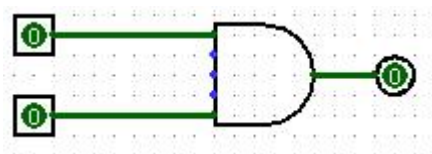


图 1-2 元件连线测试

- 5、最后，单击“Poke”按钮，试着单击电路图上的输入，看看会发生什么。这和你想象中的与门的功能相符么？

2.3 练习子电路

正如 C 程序可以包含帮助函数一样，一个电路图中也可以含有子电路。在这部分中，我们会创建几个子电路，并示范一下他们的使用。

- 1、新建一个电路图（File New）。
- 2、新建一个子电路（Project Add Circuit），并命名为 NAND。
- 3、在新电路图窗口中，你可以看见你刚创建的含有两个输入一个输出 NAND 电路。

- 4、在屏幕左侧电路选择板中双击“main”以返回主电路图。这时，最初的空白电路图会显示出来，而 NAND 电路图则被保存。
- 5、单击列表中的“NAND”，告诉 Logisim 你想吧“NAND”电路添加到主电路中。
- 6、试着把“NAND”电路放到主电路图中。如果你正确地做到了，你会看到一个左边含两个输入右边含一个输出的门。试着把输入输出相连，看看它是否和想象中一样工作。
- 7、重复这些步骤，创建其他几个子电路：NOR，XOR，2 to 1 MUX，和 4 to 1 MUX。除了 AND，OR 和 NOT 外，不要使用其他内置门。但是，一旦你创建了一个子电路，你可以使用它来创建其他电路。

3. 实验步骤

- (1) 构建一个名为“2:1 MUX”的二路选择器子电路。

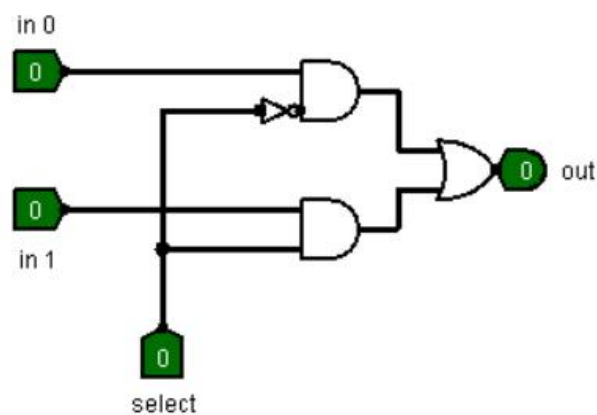


图 1-3 二路选择器电路

- (2) 在 Logisim 中使用名为“2:1 MUX”的二路选择器子电路，构建一个四路选择器。

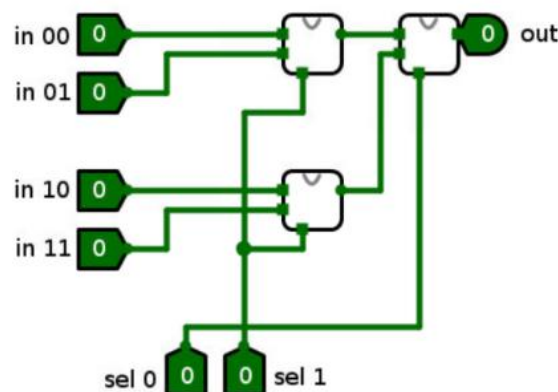


图 1-4 四路选择器电路

- (3) 编辑子电路外观：二路选择器子电路外观由矩形改成更常见的梯形外观

选择子电路，然后选择编辑子电路外观选项 ，编辑子电路的外观。

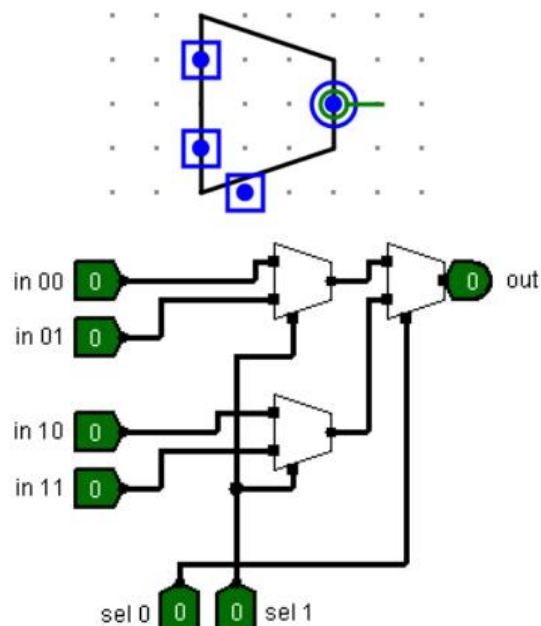


图 1-5 子电路编辑

- (4) 利用分线器（分离器）Splitter，将 8 位输入引脚中的高 4 位和低 4 位分离成两个 4 位数据，并计算他俩按位“与”的结果。

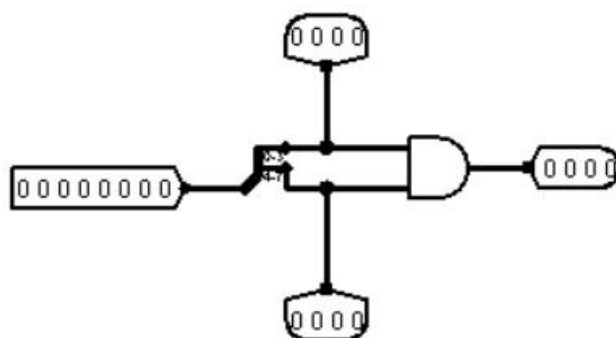
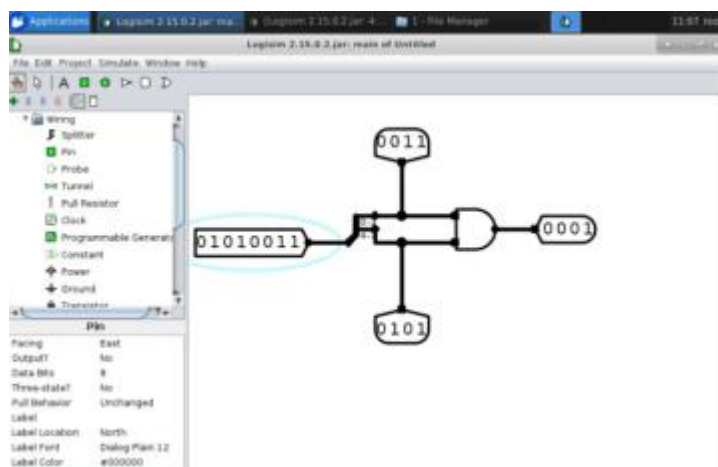


图 1-6 分线器电路

- (5) 掌握在 Logisim 中进行电路仿真测试的方法。



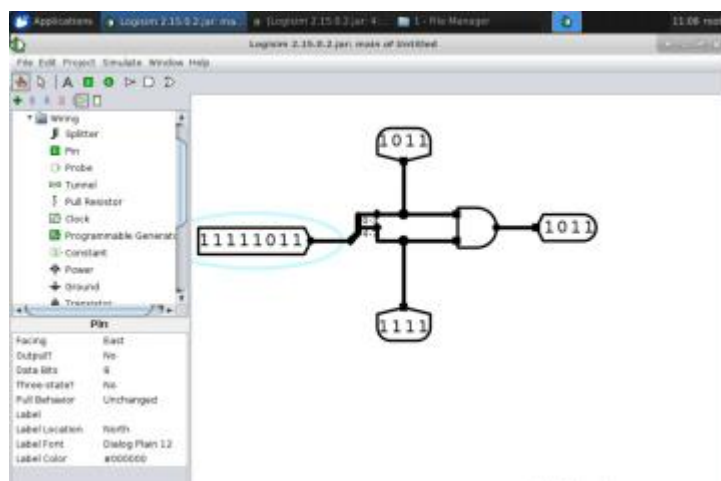


图 1-7 分线器电路测试

(6) 完成 LED 计数电路并进行测试。

- 绘制 LED 计数电路

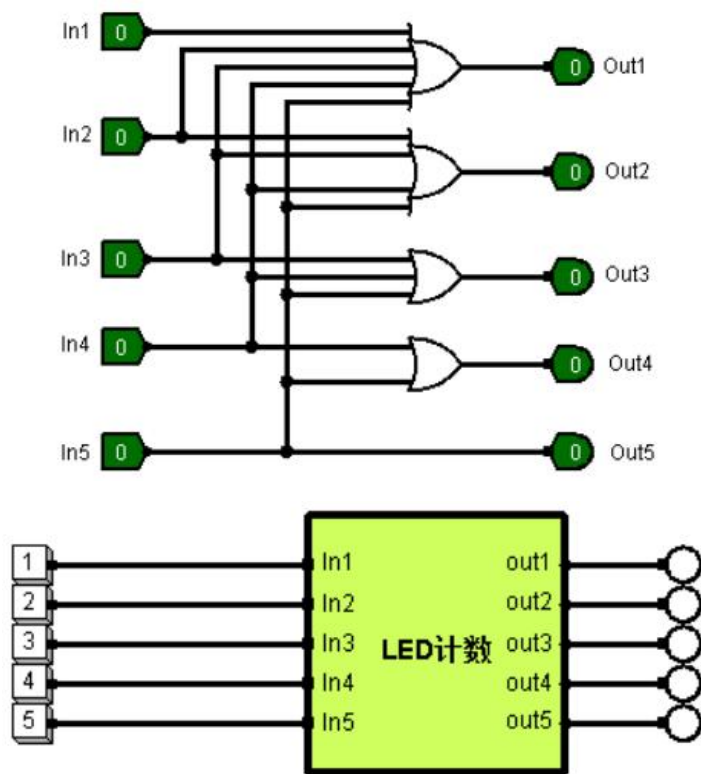


图 1-8 计数器电路

- 测试 LED 电路

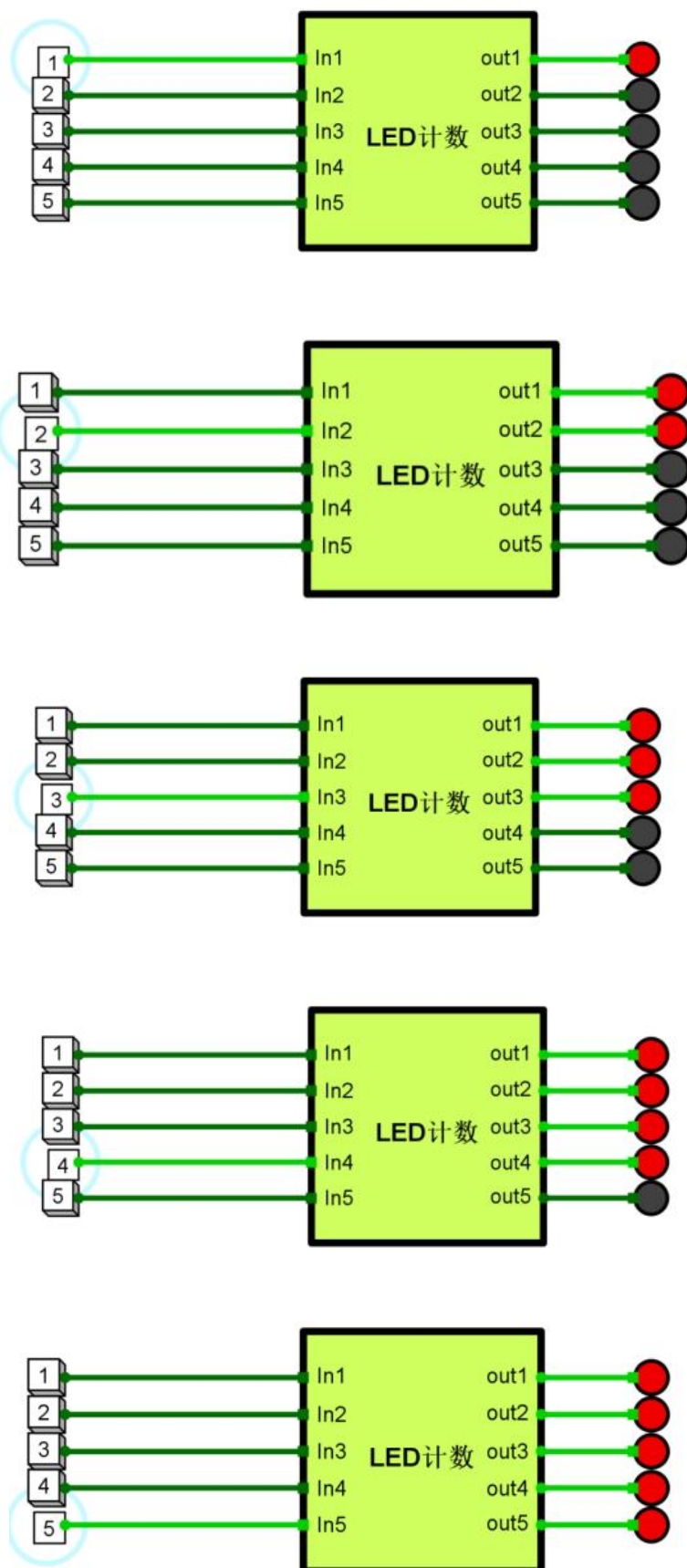


图 1-9 计数器电路测试

注：

快捷键：在封装子电路的过程中，**ctrl** 键+鼠标使每次图形绘图的顶点对齐到像素点的位置上，**shift** 键+鼠标绘制正方形，正圆形或者 45° 斜线或者标准直线。

4. 实验心得

注：可写出实验过程中出现的故障及其排除方法，实验收获等。

实验二 数据传输实验

1. 实验目的

- 1) 熟悉和了解总线的数据通路互递原理及寻址方式。
- 2) 掌握缓冲器、三态缓冲器、三态非门、奇偶校验、数据选择器、解复用器、解码器、位选择器及**优先编码器**等的基本工作原理与使用方法。

2. 实验要求

1) 三态缓冲器及三态非门

三态门也称为三态缓冲器，是指逻辑门的输出除有高、低电平两种状态外，还有第三种高阻状态的门电路，高阻态相当于隔断状态。三态缓冲器都有一个 EN 控制使能端，来控制门电路的通断。

如图 1 和图 2 所示，三态缓冲器及三态非门可用以总线方向控制。

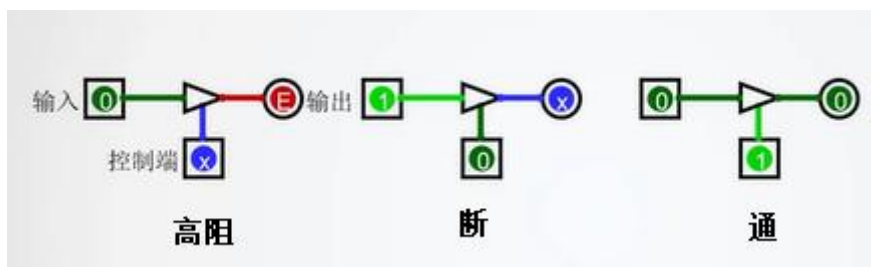


图 2-1 三态缓冲器三种控制状态图

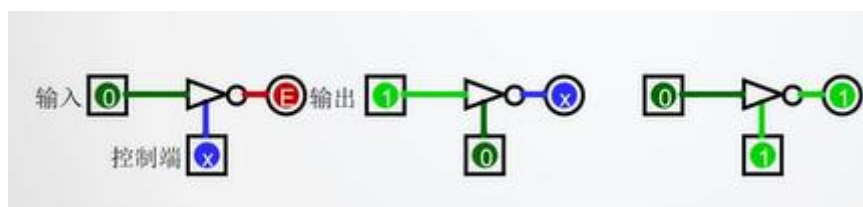


图 2 三态非门三种控制状态图

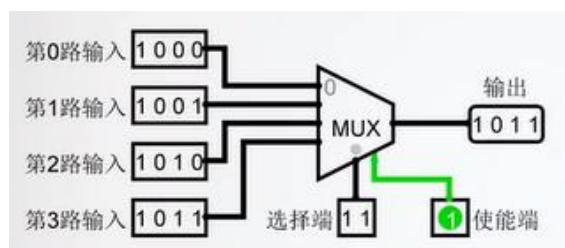


图 2-3 使能端为 1 的多路选择器

2) 多路选择器

多路选择器是数据选择器的别称，是组成原理实验中经常使用的组件。在多路数据传送过程中，能够根据需要将其中的任意一路选出来的电路，也称多路选择器或多路开关。

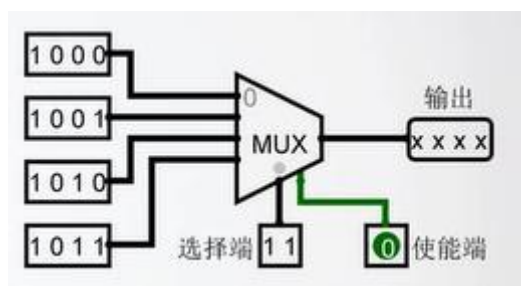


图 2-4 使能端为 0 的多路选择器

图 3 和图 4 分别为使能端为 1 和为 0 的多路选择器。

选择端位宽为 n ，则可选的输入源的数目为 2^n 。

3) 解复用器

解复用器是一个组合逻辑电路，设计用于将一条公共输入线切换到多条单独输出线中的一条数据分配器，通常称为解复用器或多输出选择器，简称为“Demux”或“DMX”，与多路选择器刚好相反。

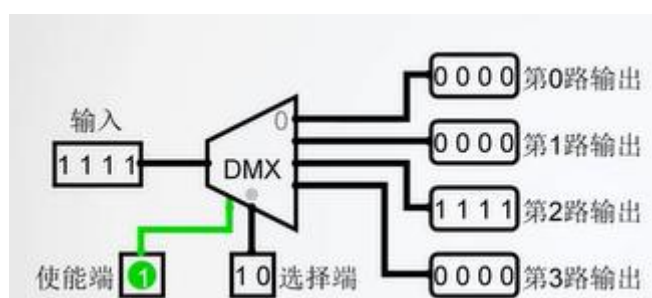


图 5 使能端为 1 的解复用器

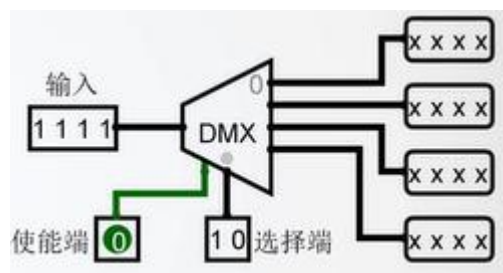


图 2-6 使能端为 0 的解复用器

图 5 和图 6 分别为使能端为 1 和为 0 的解复用器。

选择端位宽为 n ，则可选的输出通道数目为 2^n 。

4) 解码器

解码器又称为译码器，是一类多输入多输出组合逻辑电路器件，可用于地址译码。其能将输入二进制代码的各种状态，按照其原意翻译成对应的输出信号。

如，74138 是一种 3 线—8 线译码器，三个输入端 CBA 共有 8 种状态组合（000—111），可译出 8 个输出信号 Y0—Y7。

译码器设有一个和多个使能控制输入端，又成为片选端，用来控制允许译码或禁止译码。

图 7 和图 8 分别为使能端为 1 和为 0 的译码器。

n 位的选择端，对应的选择通道数为 2^n 。

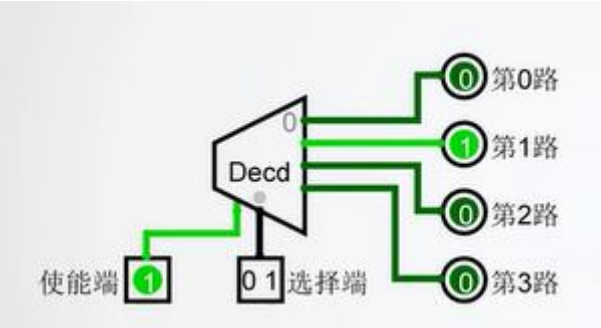


图 2-7 使能端为 1 的解码器

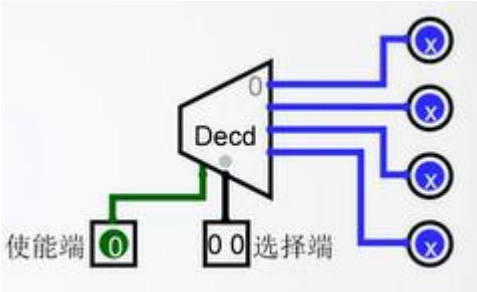


图 2-8 使能端为 1 的解码器

3. 实验步骤

3.1 三态缓冲器及三态非门

(1) 三态缓冲器及三态非门各状态控制变化表

表 3.1.1 三态缓冲器各状态控制变化表

输入	控制端	输出	状态
0	X	E	高阻
1	0	X	断
1	1	1	通

表 3.1.2 三态非门各状态控制变化表

输入	控制端	输出	状态
0	X	E	高阻
1	0	X	断
0	1	1	通

3.2 多路选择器

(1) 在理想情况下，多路选择器在不同选择位数时的输出变化

表 3.2.1 2-4 路选择器输出端变化表

输入	选择端	使能端	输出
第 0 路: 0010	10	1	1001
第 1 路: 0101	11	1	0110
第 2 路: 1001	01	1	0101
第 3 路: 0110	00	1	0010
	任何数	0	xxxx

3.3 解复用器

(1) 在理想情况下，解复用器在不同选择位数时的输出变化

表 3.3.1 解复用器在不同选择位数时的输出变化表

输入	选择端	使能端	输出
1011	11	1	第 3 路输出 1011
	10	1	第 2 路输出 1011
	00	1	第 0 路输出 1011
	随机	0	xxxx

3.4 解码器

(1) 在理想情况下，解码器在不同选择端时的输出变化

表 3.4.1 解码器在不同选择端时的输出变化表

选择端	使能端	输出
10	1	第 2 路输出 1
01	1	第 1 路输出 1
00	1	第 0 路输出 1
随机	0	都是 x

4. 实验数据

4.1 三态缓冲器及三态非门

(1) 在 logisim 中画出电路原理图

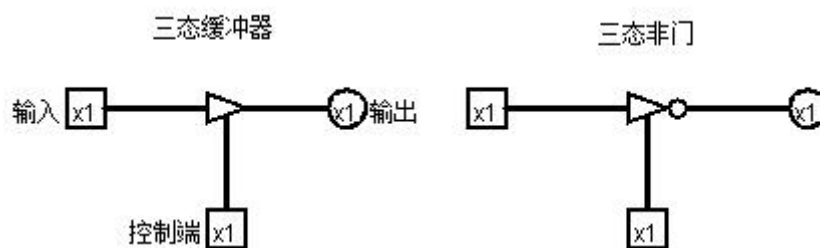


图 2-9 三态缓冲器及三态非门的电路原理图

(2) 输入数据测试实验结果

表 4.1.1 三态缓冲器测试结果

输入	控制端	输出	状态
0	X	E	高阻
1	0	X	断
1	1	1	通

表 4.1.2 三态非门测试结果

输入	控制端	输出	状态
0	X	E	高阻
1	0	X	断
0	1	1	通

4.2 多路选择器

(1) 在 logisim 中画出多路选择器的电路原理图

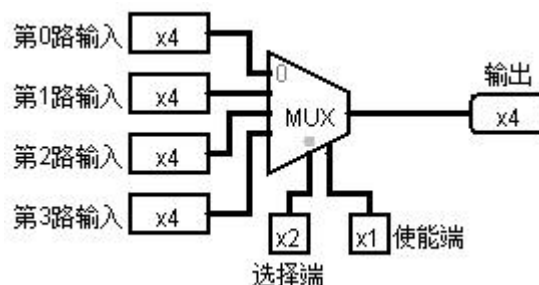


图 2-10 2-4 路选择器电路原理图

(2) 输入数据测试实验结果

表 4.2.1 2-4 路选择器输出端实验结果变化表

输入	选择端	使能端	输出
第 0 路: 0010	10	1	1001
第 1 路: 0101	11	1	0110
第 2 路: 1001	01	1	0101
第 3 路: 0110	00	1	0010
	任何数	0	xxxx

4.3 解复用器

(1) 在 logisim 中画出解复用器的电路原理图

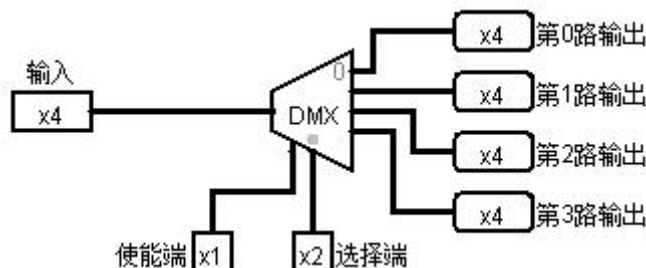


图 2-11 解复用器的电路原理图

(2) 输入数据测试实验结果

表 4.3.1 解复用器输出测试结果表

输入	选择端	使能端	输出
1011	11	1	第 3 路输出 1011
	10	1	第 2 路输出 1011
	00	1	第 0 路输出 1011
	随机	0	XXXX

4.4 解码器

(1) 在 logisim 中画出解码器的电路原理图

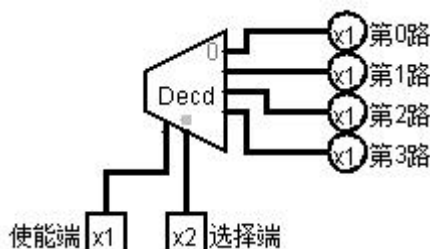


图 2-12 解码器的电路原理图

(2) 输入数据测试实验结果

表 4.4.1 解码器输入数据测试实验结果表

选择端	使能端	输出
10	1	第 2 路输出 1
01	1	第 1 路输出 1
00	1	第 0 路输出 1
随机	0	都是 x

5. 实验小结

1. 基本熟悉了 logisim 平台的操作方法。
2. 在 logisim 平台中掌握了三态缓冲器、三态非门、解复用器、解码器的原理图，并且其基本工作原理与使用方法也基本熟悉。

6. 实验心得

注：可写出实验过程中出现的故障及其排除方法，实验收获等。

实验三 数据运算实验

1. 实验目的

- (1) 完成算术运算实验，熟悉 ALU 运算器组成原理。
- (2) 在计算机中完成两个二进制数相加的基本加法器有半加器和全加器。半加器在完成两数相加时，不需要考虑低位进位。全加器用来完成两个二进制数相加，并且同时考虑低位的进位，即全加器完成三个 1 位数相加的功能。
- (3) 通过本实验理解计算机基本半加器和全加器数据运算原理。

2. 实验要求

1) 半加器

半加器电路是指对两个输入数据位相加，输出一个结果位和进位，没有进位输入的加法器电路。图 3-1 为半加器加法原理图。

图 3-2 是实现两个一位二进制数的加法运算电路。



图 3-1 半加器加法原理图

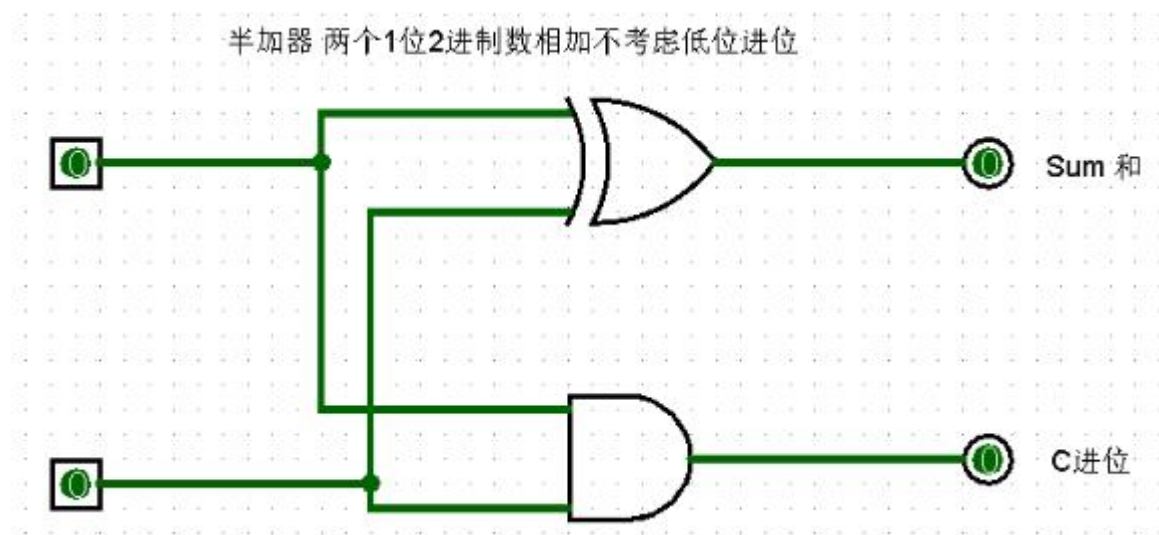


图 3-2 半加器加法运算电路

2) 全加器

用门电路实现两个二进制数相加并求出和的组合线路，称为一位全加器。图 3-3 为一位全加器加法原理。一位全加器可以处理低位进位，并输出本位加法进位。

3 个输入分别是：两个加数 A，B，前一位的进位 C。

2 个输出分别是：本位的和结果 S，下一位进位 C。

全加器真值表

输入			输出	
A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

全加器的表达式为：

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + B_i C_i + A_i C_i$$

或

$$C_i = A_i B_i + C_{i-1} (A_i + B_i)$$

或

$$C_i = A_i B_i + C_{i-1} (A_i \oplus B_i)$$

图 3-3 全加器加法原理图

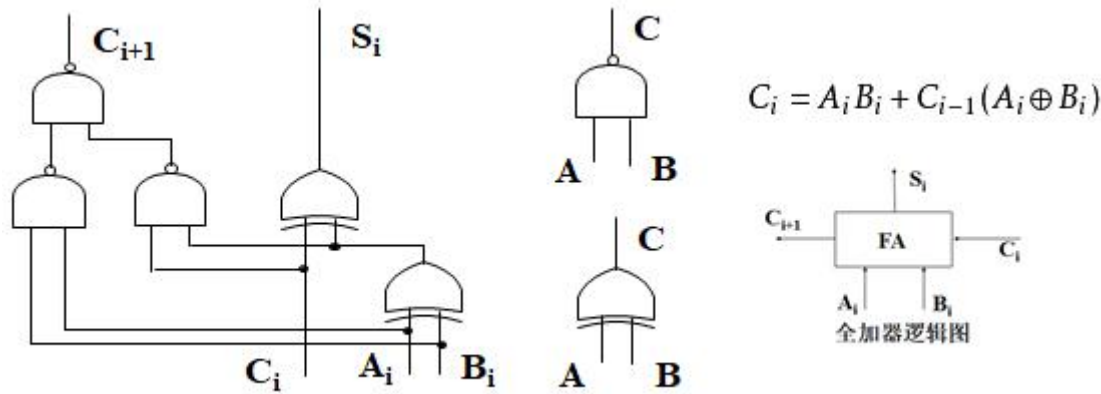


图 3-4 加法运算全加器电路

图 3-4 是实现两个一位二进制数加法运算的全加器电路。

3) 基本的二进制加法/减法器

多个一位全加器进行级联可以得到多位全加器。

如图 3-5 所示，由一位全加器，可以组成 S 位基本的二进制加法/减法器。

通过 M 控制端可以实现加法或减法，同时还可判断溢出。

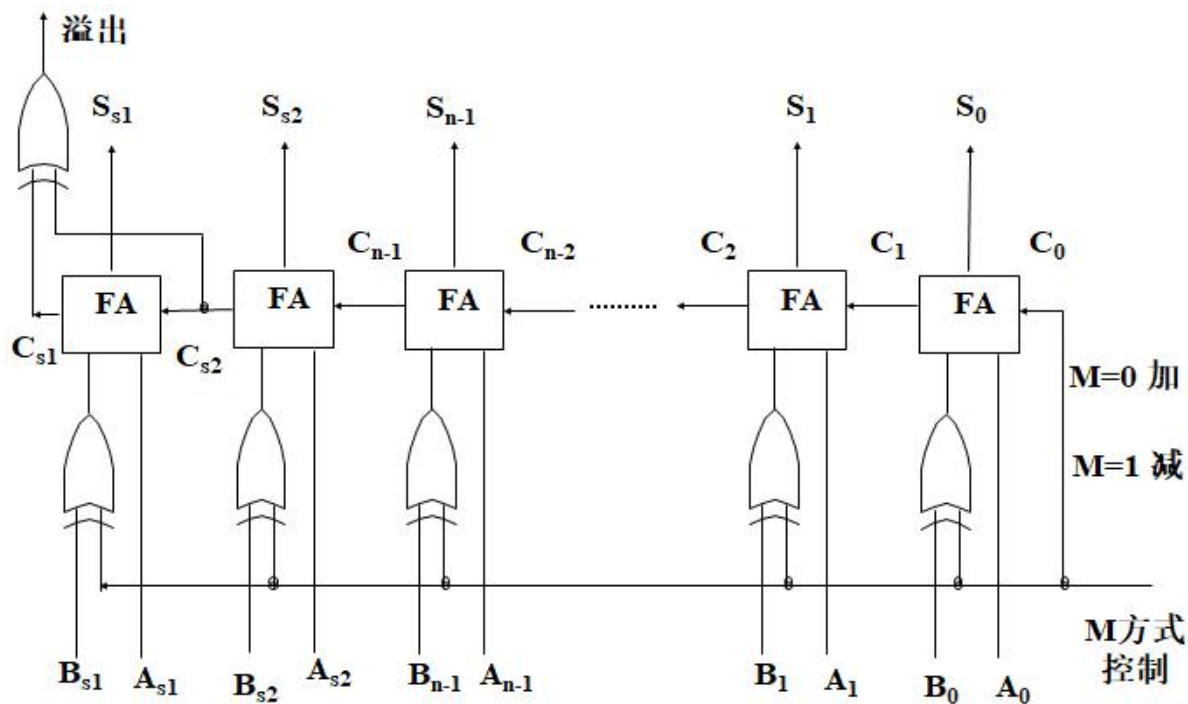


图 3-5 基本的二进制加法/减法器原理图

3. 实验步骤

3.1 半加器实验

(1) 半加器真值表

表 1 半加器真值表

输入	A _i	B _i	输出	S _i	C _i
	0	0		0	0
	0	1		1	0
	1	0		1	0
	1	1		0	1

(2) 逻辑表达式

$$S_i = A_i \oplus B_i$$

$$C_i = A_i B_i$$

(3) 在 Logisim 中画出电路图

3.2 全加器实验

(1) 全加器真值表

表 2 全加器真值表

输入	A _i	B _i	C _i	输出	S _i	C _{i+1}
	0	0	0		0	0
	0	0	1		1	0

	0	1	0		1	0
	0	1	1		0	0
	1	0	0		1	1
	1	0	1		0	0
	1	1	0		0	1
	1	1	1		1	1

(2) 逻辑表达式

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_i = A_i B_i + C(A_i \oplus B_i)$$

(3) 在 Logisim 中画出电路图

3.3 基本的二进制加法/减法器

(1) 二进制加法器/减法器真值表

表 3 二进制加法器真值表

输入						输出			
A0	B0	A1	B1	A2	B2	S0	S1	S2	溢出
0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	1	1
0	0	0	0	1	1	0	0	0	1
1	1	1	1	1	1	0	1	1	0

表 4 二进制减法器真值表

输入						输出			
A0	B0	A1	B1	A2	B2	S0	S1	S2	溢出
0	0	0	0	0	0	0	0	0	0
1	0	1	0	1	0	1	1	1	0
1	1	1	0	1	0	0	1	1	0
0	0	0	1	0	1	0	1	0	0
1	1	1	1	1	1	0	0	0	0

(2) 在 Logisim 中画出电路图

4. 实验数据

4.1 半加器实验

在 Logisim 中的电路如图 3-6 所示

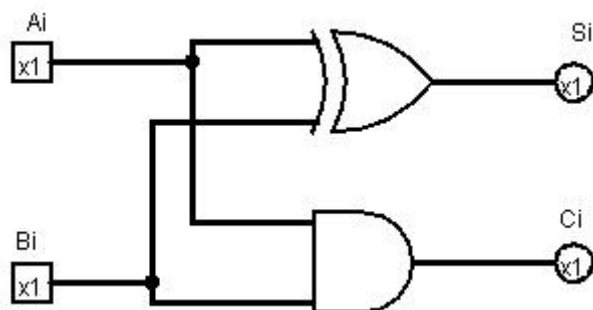


图 3-6 半加器原理图

在 Logisim 中输入数据得出的结果，如表 2 所示

表 5 半加器测试结果

输入	Ai	Bi	输出	Si	Ci
	0	0		0	0
	0	1		1	0
	1	0		1	0
	1	1		0	1

4.2 全加器实验

在 Logisim 中全加器的电路如图 3-7 所示

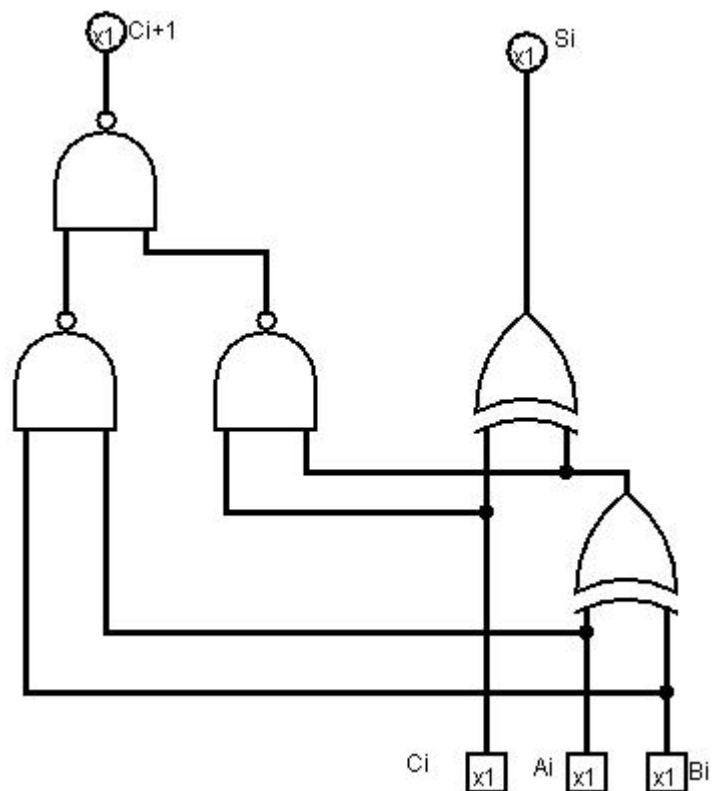


图 3-7 全加器的电路原理图

图 3-7 是实现两个一位二进制数加法运算的全加器电路，测试结果如表 3 所示

表 6 全加器测试结果表

输入	Ai	Bi	Ci	输出	Si	Ci+1
	0	0	0		0	0
	0	0	1		1	0
	0	1	0		1	0
	0	1	1		0	0
	1	0	0		1	1
	1	0	1		0	0
	1	1	0		0	1
	1	1	1		1	1

4.3 基本的二进制加法/减法器

在 Logisim 中全加器的电路如图 3-8 所示

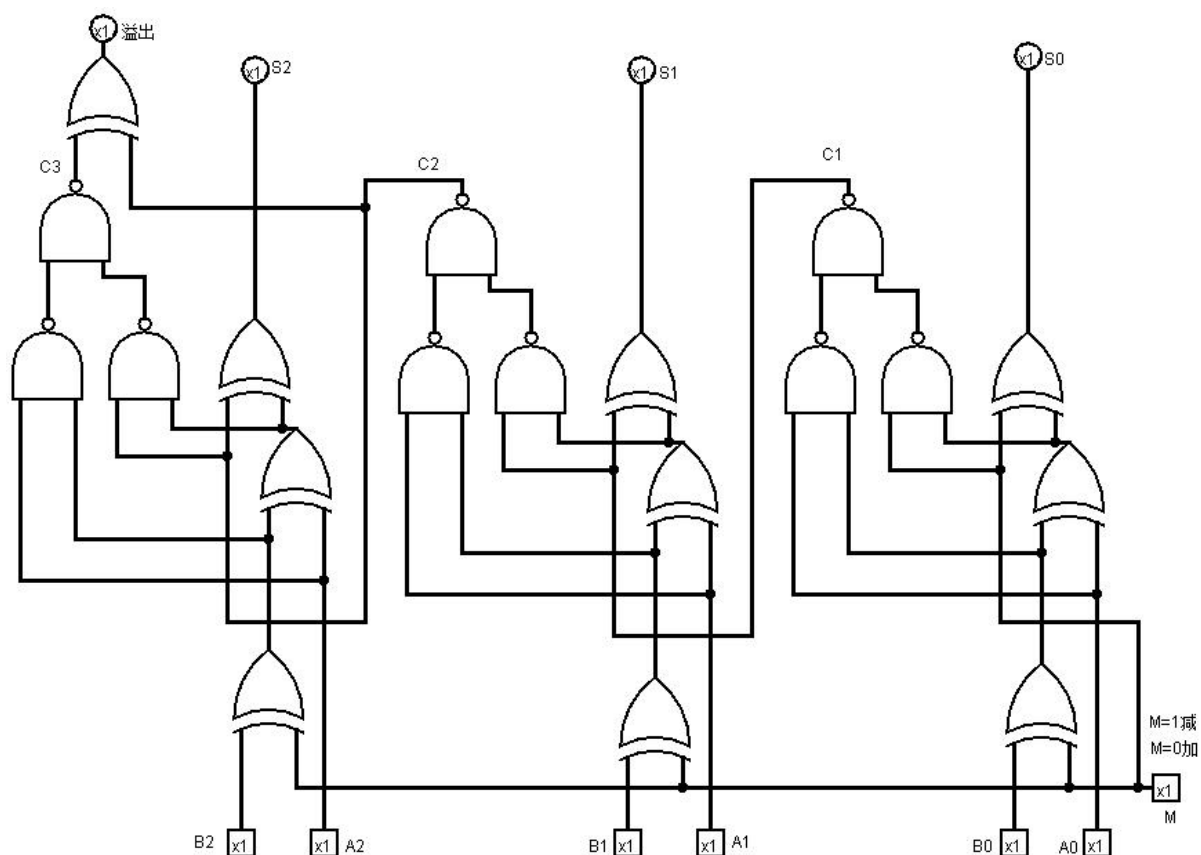


图 3-8 基本的二进制加法/减法器原理图

图 3-7 是实现两个一位二进制加法/减法器电路，测试结果如表 7 和表 8 所示

表 7 二进制加法器真值表

输入						输出			
A0	B0	A1	B1	A2	B2	S0	S1	S2	溢出
0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	1	1
0	0	0	0	1	1	0	0	0	1
1	1	1	1	1	1	0	1	1	0

表 8 二进制减法器真值表

输入						输出			
A0	B0	A1	B1	A2	B2	S0	S1	S2	溢出
0	0	0	0	0	0	0	0	0	0
1	0	1	0	1	0	1	1	1	0
1	1	1	0	1	0	0	1	1	0

0	0	0	1	0	1	0	1	0	0
1	1	1	1	1	1	0	0	0	0

5. 实验小结

- (1) 初步学会运用 Logisim 画电路原理图；
- (2) 基本理解了半加器和全加器数据运算原理。

6. 实验心得

注：可写出实验过程中出现的故障及其排除方法，实验收获等。

实验四 数据存储实验

1. 实验目的

- 1) 熟悉和了解存储器组织与总线组成的数据通路。
- 2) 掌握存储部件在计算机组成中的运用。

2. 实验要求

1) 各类触发器

触发器具有两个稳定的状态，在外加信号的触发下，可以从一个稳态翻转为另一稳态。这一新的状态在触发信号去掉后，仍然保持着，一直保留到下一次触发信号来到为止，这就是触发器的记忆作用，它可以记忆或存储两个信息：“0”或“1”。

如图 4-1 所示，常见的触发器有 D 触发器、T 触发器、JK 触发器及 RS 触发器等。

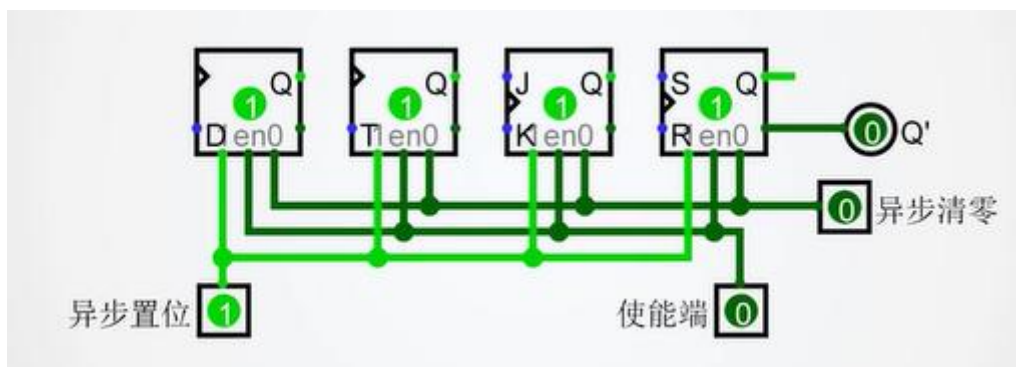


图 4-1 常见触发器状态图

2) 寄存器

寄存器的功能是存储二进制代码，它是由具有存储功能的触发器组合起来构成的。一个触发器可以存储 1 位二进制代码，故存放 n 位二进制代码的寄存器，需用 n 个触发器来构成。寄存器是中央处理器内的组成部分。寄存器是有限存储容量的高速存储部件，它们可用来暂存指令、数据和位址。

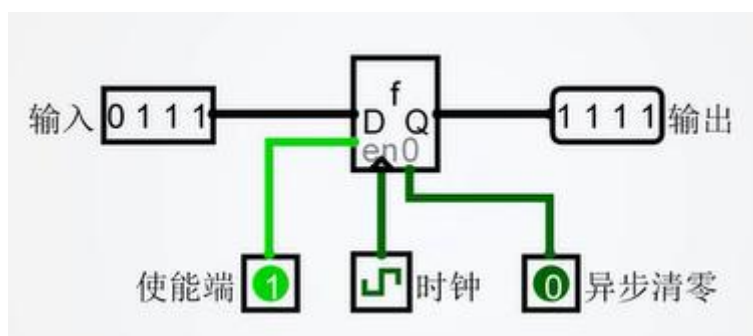


图 4-2 基本寄存器

图 4-2 为基本寄存器的组成原理图。

图 4-3 为具有同步清零和异步清零功能寄存器组成原理图。

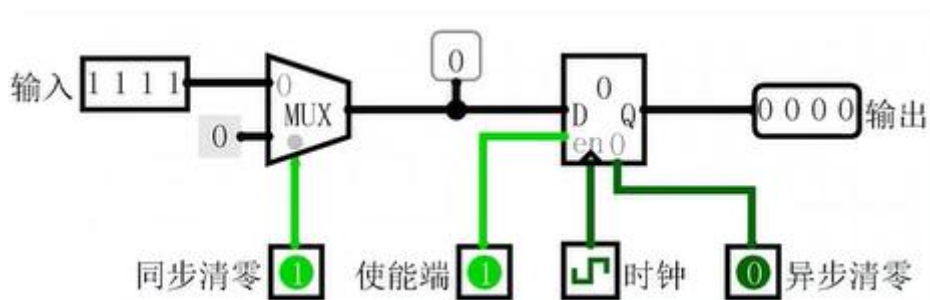


图 4-3 同步清零和异步清零寄存器

3) 计数器

计数器可实现正向和方向计数和控制功能。计数器是由基本的计数单元和一些控制门所组成，计数单元则由一系列具有存储信息功能的各类触发器构成，这些触发器有 RS 触发器、T 触发器、D 触发器及 JK 触发器等。

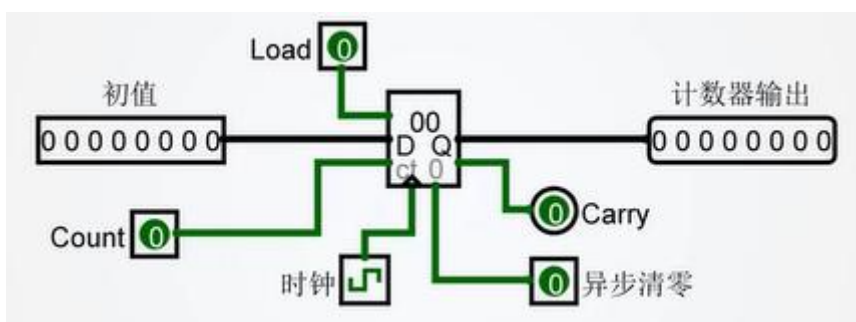


图 4 基本计数器

图 4 为基本计数器的组成原理图。

4) 移位寄存器

移位寄存器不仅能寄存数据，而且能在时钟信号的作用下使其中的数据依次左移或右移。移位寄存器可以用来寄存代码，还可以用来实现数据的串行—并行转换、数值的运算以及数据的处理等。

图 4-5 为基本移位寄存器组成原理图。

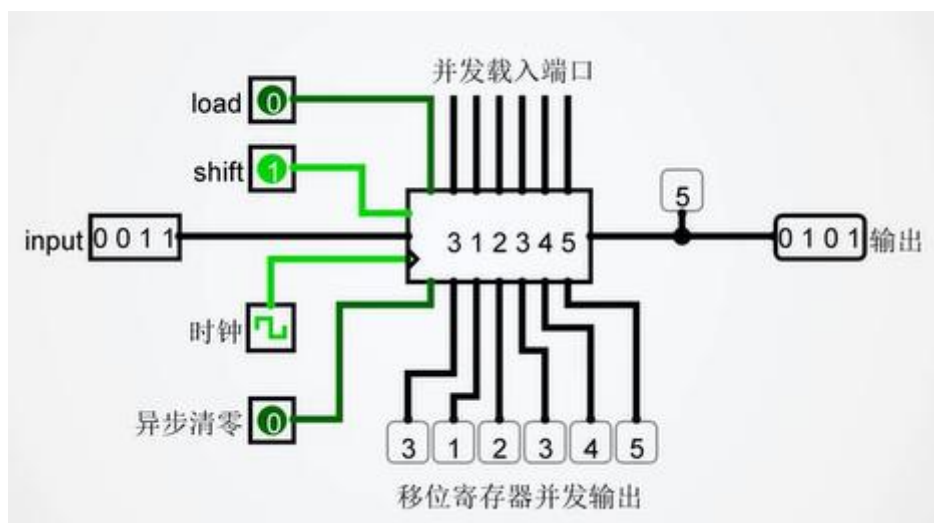


图 5 基本移位寄存器

5) ROM

只读存储器（ROM）是一种在正常工作时其存储的数据固定不变，其中的数据只能读出，不能写入，即使断电也能够保留数据，要想在只读存储器中存入或改变数据，必须具备特定的条件。

图 4-6 为基本 ROM 存储器组成原理图。

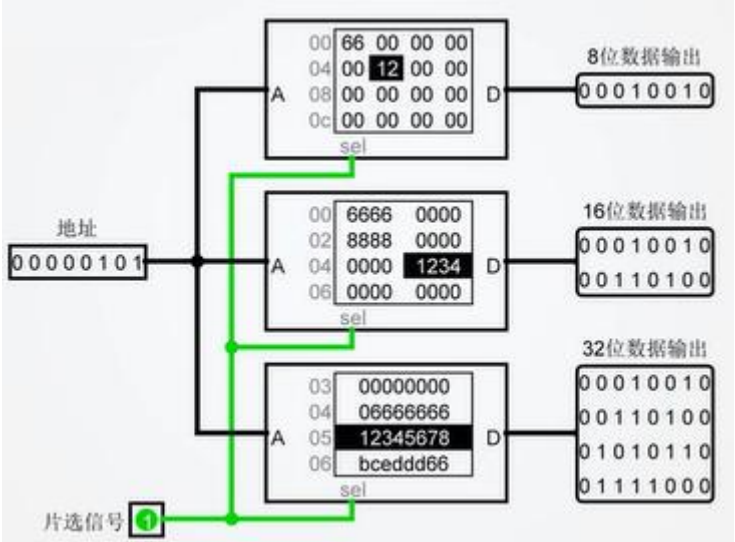


图 4-6 基本 ROM 存储器

6) RAM

随机存取存储器（RAM）又可称为读写存储器，它不仅可以存储大量的信息，而且在操作过程中能任意“读取”某个单元信息，或在某个单元中“写入”需要存储的信息，当电源关闭时随机存储器不能保留所存储的数据。

图 4-7 为 Logisim 中三种基本 RAM 存储器。

图 4-8 为 Logisim 中三种基本 RAM 存储器选择界面。



图 4-7 三种基本 RAM 存储器

选区：随机存储器 (RAM)	
地址位宽	8
数据位宽	8
数据接口	一个同步加载/存储引脚
	一个同步加载/存储引脚
	一个异步加载/存储引脚
	分离的加载和存储引脚

图 4-8 三种基本 RAM 存储器选择

图 4-9 为双向输入输出同步模式的 RAM 存储器组成原理。

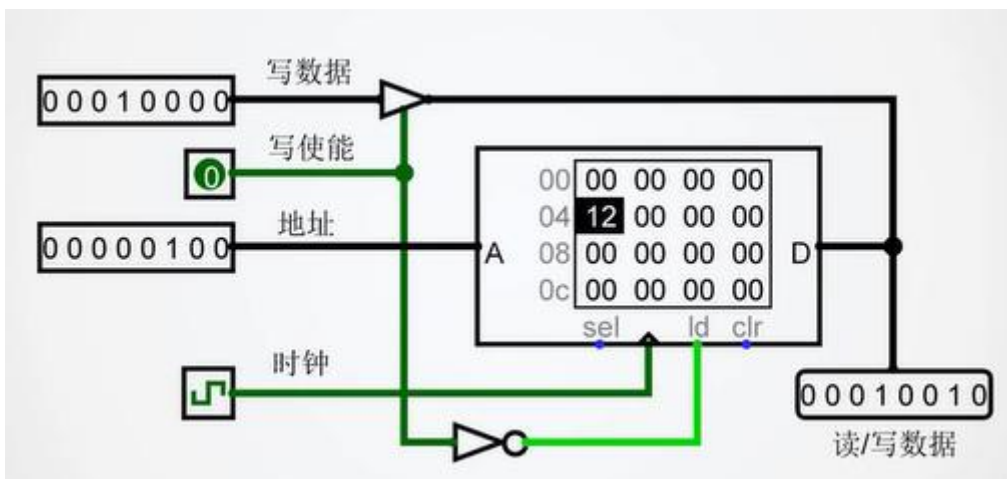


图 4-9 双向输入输出同步模式 RAM 存储器

图 10 为输入输出分离同步模式的 RAM 存储器组成原理。

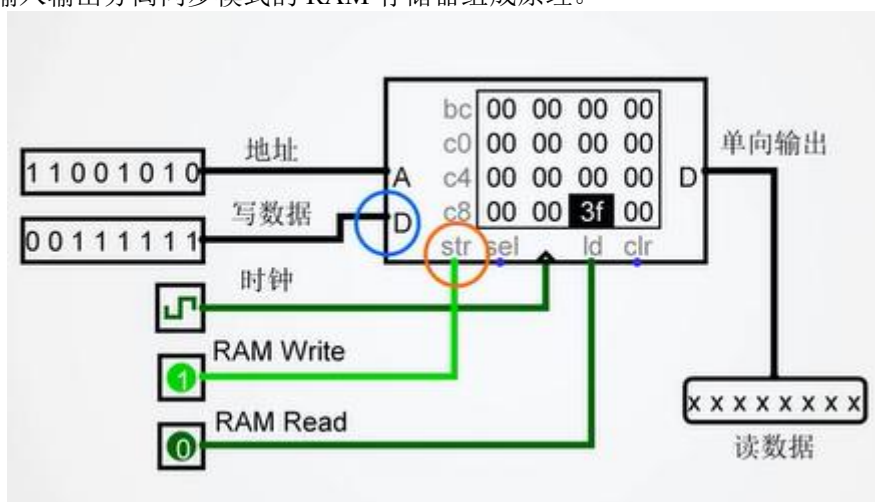


图 4-10 输入输出分离同步模式 RAM 存储器

3. 实验步骤

3.1 各类触发器

(1) 将 D 触发器、T 触发器、JK 触发器及 RS 触发器连接在一起，异步位置、异步清零、使能端及 Q' 的数值变化如下表。

表 3.1 触发器数值变化表

输入			触发器				输出
异步位置	异步清零	使能端	D 触发器	T 触发器	JK 触发器	RS 触发器	Q'
1	0	0	1	1	1	1	0
0	0	0	1	1	1	1	0
0	1	0	0	0	0	0	1

3.2 寄存器

(1) 了解基本寄存器的组成原理，理论输入、输出数值变化表如下。

表 3.2 基本寄存器的输入、输出数值变化表

输入	使能端	异步清零	输出
0011	1	0	0011
0010	1	0	0010
0101	1	0	0101
1110	1	0	1110

3.3 计数器

(1) 在基本计数器下输入、输出数值变化表如下。

表 3.3 基本计数器的输入、输出数值变化表

初值	Count	时钟次数	计数器输出
00000000	1	1	00000001
00000000	1	3	00000011
00000000	1	6	00000110

3.4 移位寄存器

(1) 了解基本移位寄存器的组成原理，在时钟信号下理论输入、输出数值变化表如下。

表 3.4 移位寄存器的输入、输出数值变化表

输入	并发输出						输出
0001	1	0	0	0	0	0	0000
0011	3	1	0	0	0	0	0000
0010	2	3	1	0	0	0	0000
0110	6	2	3	1	0	0	0000
0111	7	6	2	3	1	0	0000
0101	5	7	6	2	3	1	0000
0100	4	5	7	6	2	3	0011

3.5 ROM

(1) 了解只读存储器（ROM）的组成原理，并在 Logisim 画出 ROM 的电路原理图。

3.6 RAM

(1) 了解随机存取存储器（RAM）的组成原理，并在 Logisim 画出 RAM 的电路原理图。

4. 实验数据

4.1 各类触发器

(1) 在 Logisim 中画出电路图

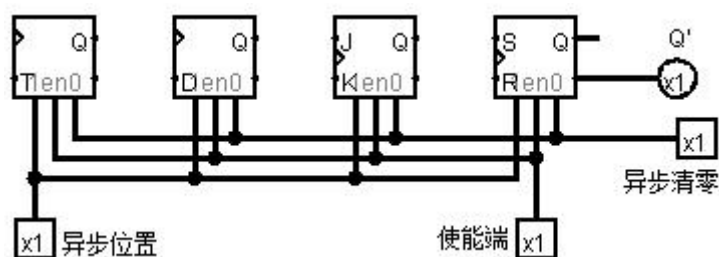


图 4-11 各类触发器连接图

(2) 输入数值

表 4.1 触发器数值变化表

输入			触发器				输出
异步位置	异步清零	使能端	D 触发器	T 触发器	JK 触发器	RS 触发器	Q'
1	0	0	1	1	1	1	0
0	0	0	1	1	1	1	0
0	1	0	0	0	0	0	1

4.2 寄存器

(1) 在 Logisim 中画出基本寄存器的电路图

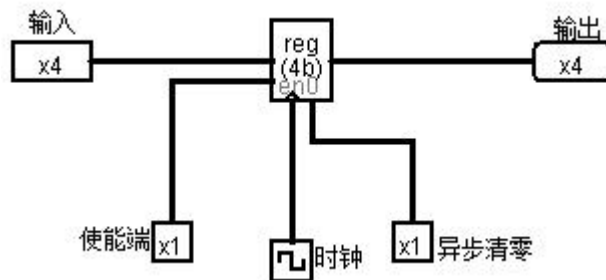


图 4-12 基本寄存器的电路原理图

(2) 输入数值变化

表 4.2 基本寄存器的输入、输出数值变化表

输入	使能端	异步清零	输出
0011	1	0	0011
0010	1	0	0010
0101	1	0	0101
1110	1	0	1110

4.3 计数器

(1) 在 Logisim 中画出基本计数器的电路图

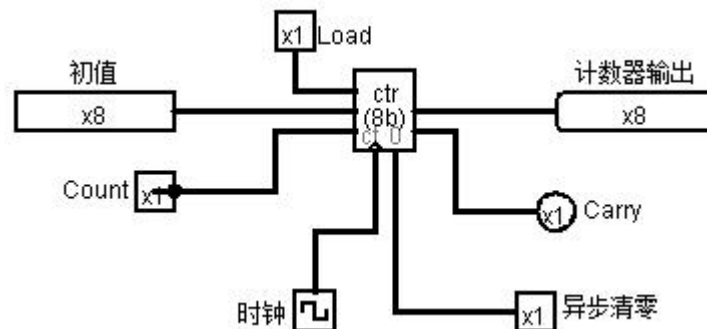


图 4-13 基本计数器的电路原理图

(2) 输入数值变化

表 4.3 基本计数器的输入、输出数值变化表

初值	Count	时钟次数	计数器输出
00000000	1	1	00000001
00000000	1	3	00000011
00000000	1	6	00000110

4.4 移位寄存器

(1) 在 Logisim 中画出基本移位寄存器的电路图

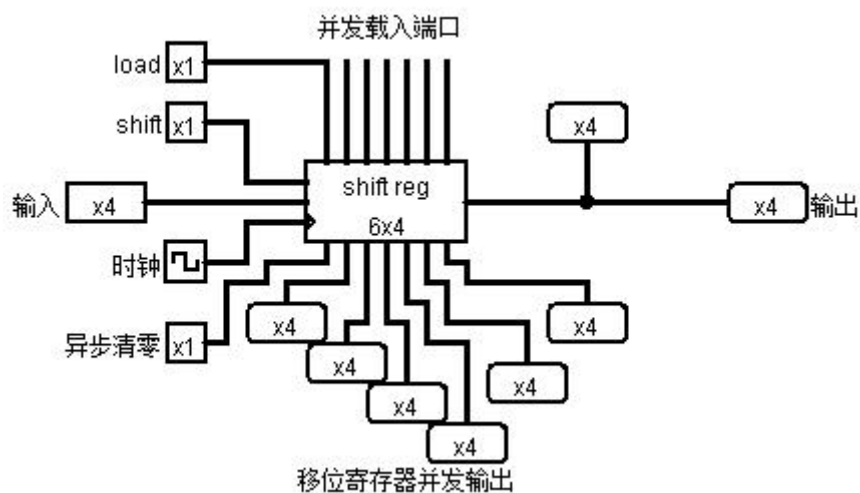


图 4-14 移位寄存器的电路原理图

(2) 输入数值测试结果变化

表 4.4 移位寄存器的测试结果表

输入	并发输出						输出
0001	1	0	0	0	0	0	0000
0011	3	1	0	0	0	0	0000
0010	2	3	1	0	0	0	0000
0110	6	2	3	1	0	0	0000
0111	7	6	2	3	1	0	0000
0101	5	7	6	2	3	1	0000
0100	4	5	7	6	2	3	0011

4.5 只读存储器（ROM）

在 Logisim 中画出基本 ROM 的电路原理图

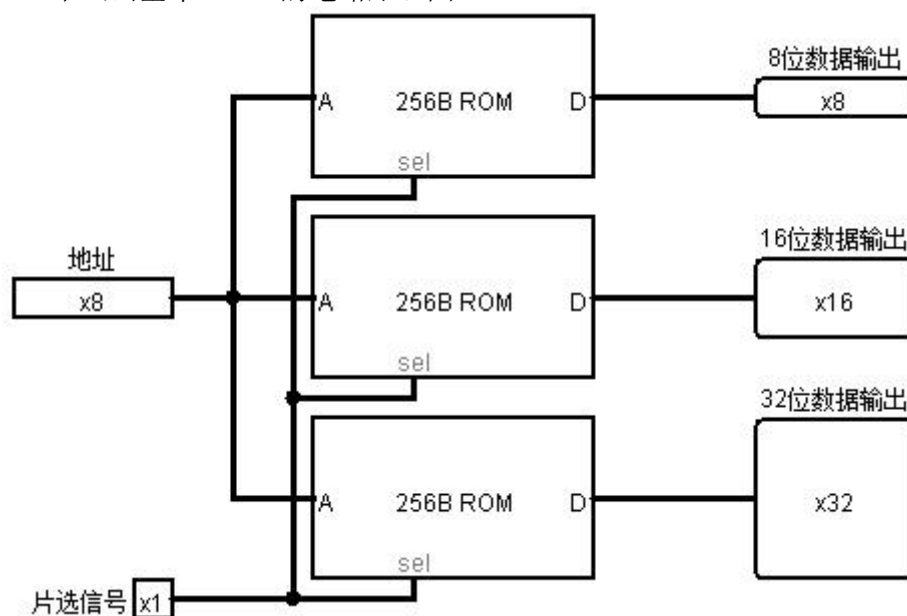


图 4-15 基本 ROM 电路原理图

4.6 随机存取存储器（RAM）

在 Logisim 中画出基本 RAM 的电路原理图

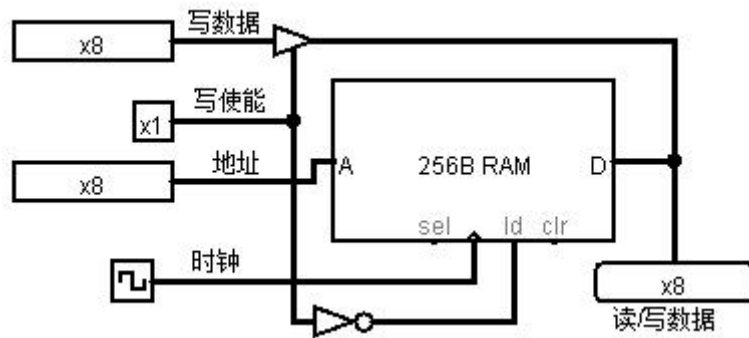


图 4-16 RAM 电路原理图

5. 实验小结

(1) 了解了存储器组织与总线组成的数据通路，并且熟悉了寄存器以及 ROM 和 RAM 的工作原理。

(2) 基本掌握了各储存部件在计算机中的运用方法。

6. 实验心得

注：可写出实验过程中出现的故障及其排除方法，实验收获等。

实验五 利用 DEBUG 调试汇编语言程序段

一. 实验目的

1. 熟悉 DEBUG 有关命令的使用方法;
2. 利用 DEBUG 掌握有关指令的功能;
3. 利用 DEBUG 运行简单的程序段。

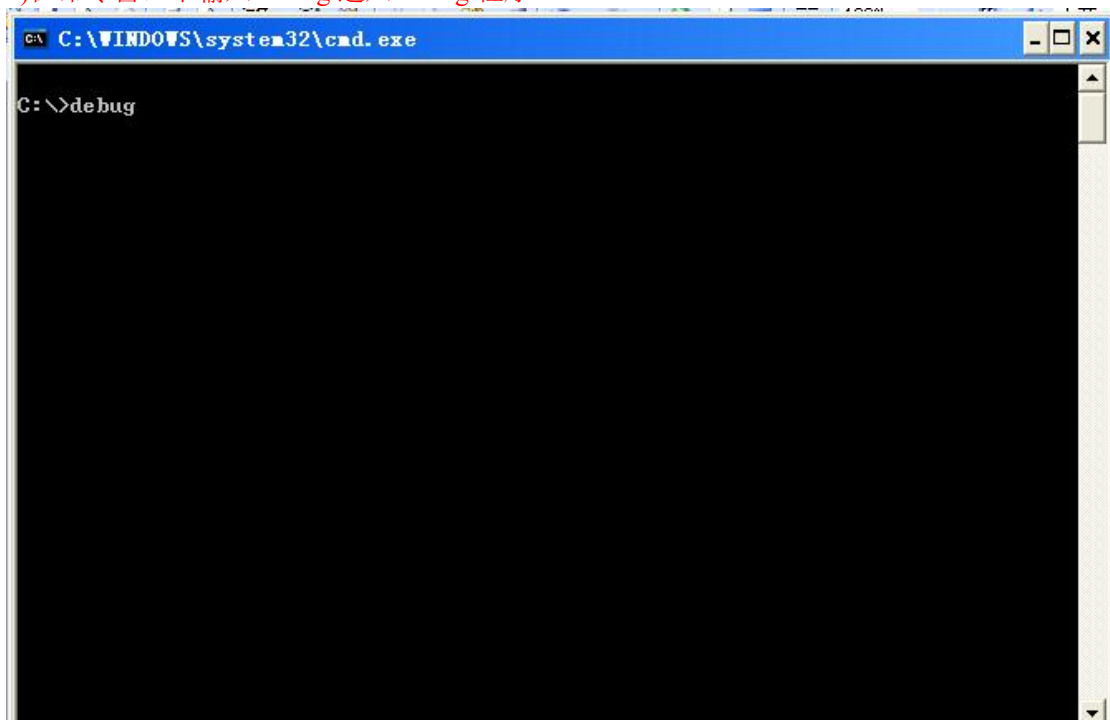
二. 实验内容

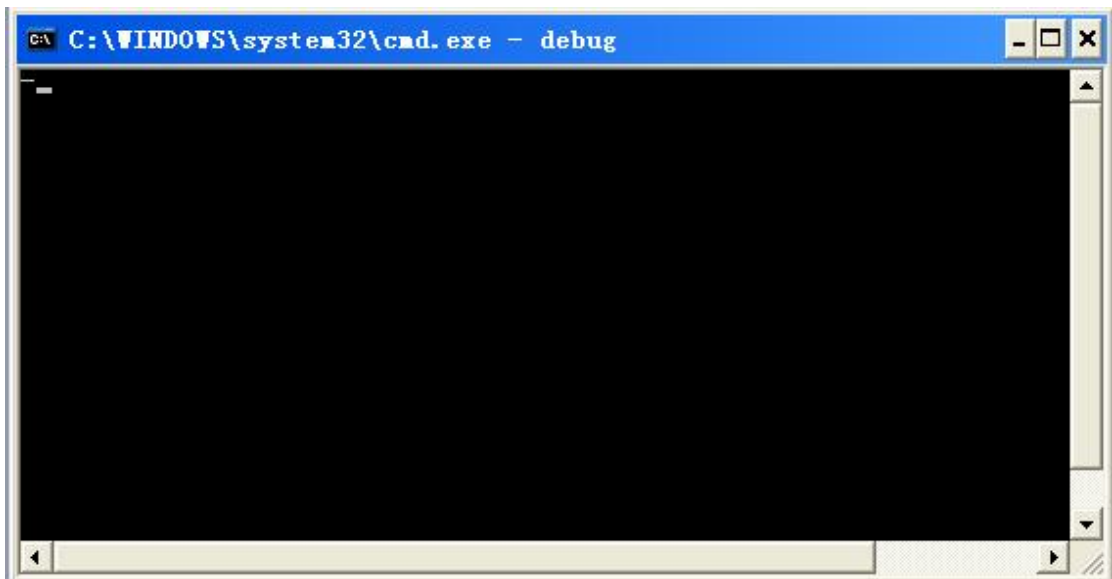
1. 进入和退出 DEBUG 程序;

1)开始—运行,输入 cmd,点确定进入命令窗口

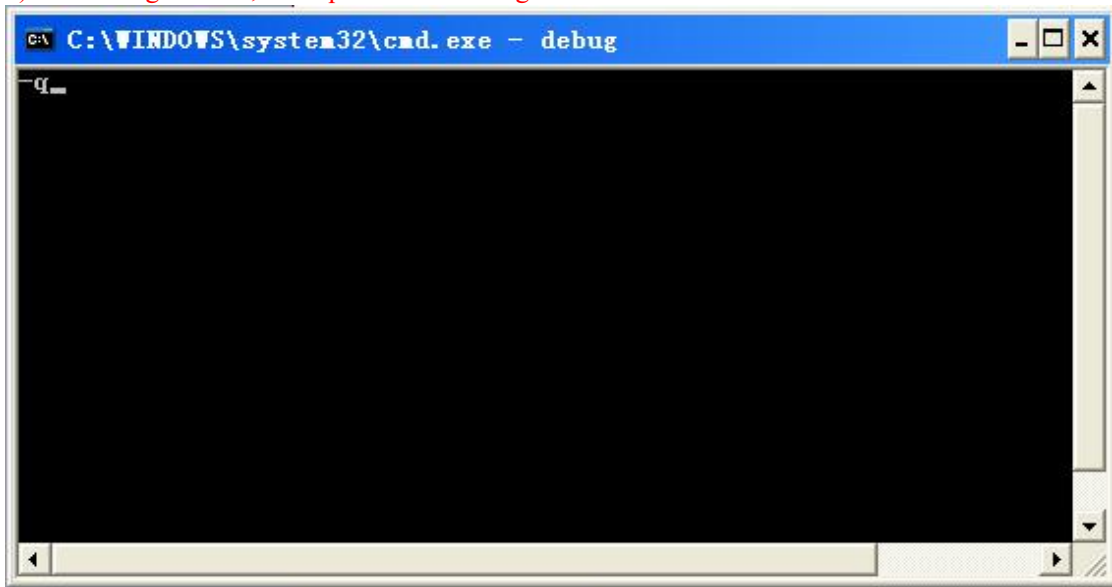


2)在命令窗口中输入 debug 进入 debug 程序





3) 进入 debug 窗口后, 输入 q 命令退出 debug



2. 学会 DEBUG 中的

1) D 命令 (显示内存数据 D 段地址: 偏移地址)

例 1: -D100 ; 显示 DS 段, 0100 开始的 128 个字节内容

```
-d100
0B43:0100  40 EB CE E8 2B 00 E8 B4-DF 06 57 51 0E 07 BF A3  @...+...WQ...
0B43:0110  8F B9 06 00 81 CD 00 40-F2 AE 75 0B 34 00 32 0B  .....@..u.4.2.
0B43:0120  B8 01 00 D3 E0 0B E8 59-5F 07 B0 00 AA 5F 9D F8  .....Y_...
0B43:0130  C3 AA 41 FE 06 E8 99 C3-2E C7 06 55 91 00 00 2E  ..A.....U...
0B43:0140  89 0E DF 91 2E 89 26 E1-91 2E 89 36 E3 91 FC 2E  .....&...6...
0B43:0150  89 0E 48 91 2E C7 06 4A-91 00 00 2E C7 06 5D 91  ..H...J.....l.
0B43:0160  00 00 2E C7 06 4E 91 00-00 2E C7 06 1A 92 5B 5D  .....N.....[ ]
0B43:0170  2E C7 06 1C 92 7C 3C 2E-C7 06 1E 92 3E 2B 2E C7  .....!<.....>+..
```

说明: 指定要显示其内容的内存区域的起始和结束地址, 或起始地址和长度。

① D SEGREG[起始地址] [L 长度]

; 显示 SEGREG 段中 (缺省内默认为 DS), 以 [起始地址] (缺省内为当前的偏移地址),

开始的 [L 长度] (缺省内默认为 128) 个字节的内容。

② D SEGREG[段地址: 偏移地址]

;显示 SEGREG 段中(缺省内默认为 DS), [段地址:偏移地址] 开始的[L 长度] (缺省内默认为 128)个字节内容

—D ;默认段寄存器为 DS, 当前偏移地址(刚进入 debug 程序偏移地址为 0100H)

```
C:\>debug
-d
0B43:0100  40 EB CE E8 2B 00 E8 B4-DF 06 57 51 0E 07 BF A3  @...+.....WQ....
0B43:0110  8F B9 06 00 81 CD 00 40-F2 AE 75 0B 34 00 32 0B  .....@..u.4.2.
0B43:0120  B8 01 00 D3 E0 0B E8 59-5F 07 B0 00 AA 5F 9D F8  .....Y.....
0B43:0130  C3 AA 41 FE 06 E8 99 C3-2E C7 06 55 91 00 00 2E  ..A.....U....
0B43:0140  89 0E DF 91 2E 89 26 E1-91 2E 89 36 E3 91 FC 2E  ....&.....6....
0B43:0150  89 0E 48 91 2E C7 06 4A-91 00 00 2E C7 06 5D 91  ..H....J.....l.
0B43:0160  00 00 2E C7 06 4E 91 00-00 2E C7 06 1A 92 5B 5D  ....N.....[ ]
0B43:0170  2E C7 06 1C 92 7C 3C 2E-C7 06 1E 92 3E 2B 2E C7  ....!<.....>+..
```

—D DS:100 ;显示 DS 段, 0100H 开始的 128 个字节内容

```
-d ds:100
0B43:0100  40 EB CE E8 2B 00 E8 B4-DF 06 57 51 0E 07 BF A3  @...+.....WQ....
0B43:0110  8F B9 06 00 81 CD 00 40-F2 AE 75 0B 34 00 32 0B  .....@..u.4.2.
0B43:0120  B8 01 00 D3 E0 0B E8 59-5F 07 B0 00 AA 5F 9D F8  .....Y.....
0B43:0130  C3 AA 41 FE 06 E8 99 C3-2E C7 06 55 91 00 00 2E  ..A.....U....
0B43:0140  89 0E DF 91 2E 89 26 E1-91 2E 89 36 E3 91 FC 2E  ....&.....6....
0B43:0150  89 0E 48 91 2E C7 06 4A-91 00 00 2E C7 06 5D 91  ..H....J.....l.
0B43:0160  00 00 2E C7 06 4E 91 00-00 2E C7 06 1A 92 5B 5D  ....N.....[ ]
0B43:0170  2E C7 06 1C 92 7C 3C 2E-C7 06 1E 92 3E 2B 2E C7  ....!<.....>+..
```

—D CS:200 ;显示 CS 段, 0200H 开始的 128 个字节内容

```
-d cs:200
0B43:0200  5B 91 26 8B 1D 8D 36 5F-91 2E 80 3C 2F 74 36 2E  [.&...6_...</t6.
0B43:0210  80 3C 22 74 08 2E F6 06-56 91 01 75 54 26 8A 47  <"t....U...uT&.G
0B43:0220  01 32 E4 2E 39 06 48 91-73 12 2E A1 48 91 D1 E0  .2..9.H.s...H...
0B43:0230  43 43 03 D8 26 8B 1F E8-88 00 EB 69 2E C7 06 4A  CC..&.....i...J
0B43:0240  91 01 00 EB 60 26 8A 47-01 32 E4 40 D1 E0 03 D8  ....'&.G.2.@....
0B43:0250  26 8A 0F 32 ED 0B C9 74-0F 43 53 26 8B 1F E8 C6  &..2...t.CS&....
0B43:0260  00 5B 73 41 43 43 E2 F2-2E C7 06 4A 91 03 00 EB  .[sACC.....J....
0B43:0270  34 26 8A 47 01 32 E4 40-D1 E0 03 D8 26 8A 07 32  4&.G.2.@....&..2
```

—D 200:100 ;显示 DS 段, 0200:0100H 开始的 128 个字节内容

```
-d 200:100
0200:0100  28 18 10 00 08 08 03 00-02 67 2D 27 28 90 2B A0  <.....g-'<.+..
0200:0110  BF 1F 00 4F 0D 0E 00 00-00 00 9C AE 8F 14 1F 96  ...0.....
0200:0120  B9 A3 FF 00 01 02 03 04-05 14 07 38 39 3A 3B 3C  ...9.....89;;<
0200:0130  3D 3E 3F 0C 00 0F 08 00-00 00 00 00 10 0E 00 FF  =>?.....
0200:0140  50 18 10 00 10 00 03 00-02 67 5F 4F 50 82 55 81  P.....g_OP.U.
0200:0150  BF 1F 00 4F 0D 0E 00 00-00 00 9C 8E 8F 28 1F 96  ...0.....<...
0200:0160  B9 A3 FF 00 01 02 03 04-05 14 07 38 39 3A 3B 3C  ...9.....89;;<
0200:0170  3D 3E 3F 0C 00 0F 08 00-00 00 00 00 10 0E 00 FF  =>?.....
```

—D 200 ; 显示 DS 段, 0200H 开始的 128 个字节内容

```
-d 200
0B43:0200  5B 91 26 8B 1D 8D 36 5F-91 2E 80 3C 2F 74 36 2E  [.&...6_...</t6.
0B43:0210  80 3C 22 74 08 2E F6 06-56 91 01 75 54 26 8A 47  <"t....U...uT&.G
0B43:0220  01 32 E4 2E 39 06 48 91-73 12 2E A1 48 91 D1 E0  .2..9.H.s...H...
0B43:0230  43 43 03 D8 26 8B 1F E8-88 00 EB 69 2E C7 06 4A  CC..&.....i...J
0B43:0240  91 01 00 EB 60 26 8A 47-01 32 E4 40 D1 E0 03 D8  ....'&.G.2.@....
0B43:0250  26 8A 0F 32 ED 0B C9 74-0F 43 53 26 8B 1F E8 C6  &..2...t.CS&....
0B43:0260  00 5B 73 41 43 43 E2 F2-2E C7 06 4A 91 03 00 EB  .[sACC.....J....
0B43:0270  34 26 8A 47 01 32 E4 40-D1 E0 03 D8 26 8A 07 32  4&.G.2.@....&..2
```

—D 100 L 10 ;显示 DS 段, 100H 开始的 100H 个字节内容

```
-d 100 L 10
0B43:0100  40 EB CE E8 2B 00 E8 B4-DF 06 57 51 0E 07 BF A3  @...+.....WQ....
```

2) E 命令 (修改指定内存)

例 1: -E100 41 42 43 44 48 47 46 45

-D 100, L08

结果: 08F1: 0100 41 42 43 44 48 47 46 45

例 2: -E 100

08F1: 0100 76 42 : 42 是操作员键入

此命令是将原 100 号内存内容 76 修改为 42，用 D 命令可察看。

① E 地址：从指定地址开始，修改（或连续修改）存储单元内容。DEBUG 首先显示指定单元内容，如要修改，可输入新数据；空格键显示下一个单元内容并可修改，减号键显示上一个单元内容并可修改；如不修改，可直接按空格键或减号键；回车键结束命令。

② E 地址 数据表：从指定的地址开始用数据表给定的数据修改存储单元。

-E DS:100 F3 'AB' 8D；把 DS 段中 0100H 开始的四个字节修改为 F3 'AB'(A 和 B 的 ASCII 码) 8D

```
-d ds:100
0B43:0100 00 00 00 00 2B 00 E8 B4-DF 06 57 51 0E 07 BF A3
0B43:0110 8F B9 06 00 81 CD 40-F2 AE 75 0B 34 00 32 0B
0B43:0120 B8 01 00 D3 E0 0B E8 59-5F 07 B0 00 AA 5F 9D F8
0B43:0130 C3 AA 41 FE 06 E8 99 C3-2E C7 06 55 91 00 00 2E
0B43:0140 89 0E DF 91 2E 89 26 E1-91 2E 89 36 E3 91 FC 2E
0B43:0150 89 0E 48 91 2E C7 06 4A-91 00 00 2E C7 06 5D 91
0B43:0160 00 00 2E C7 06 4E 91 00-00 2E C7 06 1A 92 5B 5D
0B43:0170 2E C7 06 1C 92 7C 3C 2E-C7 06 1E 92 3E 2B 2E C7
-e ds:100 f3'AB'8d
-d ds:100
0B43:0100 F3 41 42 8D 2B 00 E8 B4-DF 06 57 51 0E 07 BF A3
0B43:0110 8F B9 06 00 81 CD 40-F2 AE 75 0B 34 00 32 0B
0B43:0120 B8 01 00 D3 E0 0B E8 59-5F 07 B0 00 AA 5F 9D F8
0B43:0130 C3 AA 41 FE 06 E8 99 C3-2E C7 06 55 91 00 00 2E
0B43:0140 89 0E DF 91 2E 89 26 E1-91 2E 89 36 E3 91 FC 2E
0B43:0150 89 0E 48 91 2E C7 06 4A-91 00 00 2E C7 06 5D 91
0B43:0160 00 00 2E C7 06 4E 91 00-00 2E C7 06 1A 92 5B 5D
0B43:0170 2E C7 06 1C 92 7C 3C 2E-C7 06 1E 92 3E 2B 2E C7
```

也可以按下面的方式实现

```
-e ds:100
0B43:0100 00.f3 00.41 00.42 00.8d
```

3) R 命令（显示当前寄存器的内容）

显示修改寄存器命令 R

R: ★显示所有寄存器和标志位状态；

★显示当前 CS: IP 指向的指令。

显示标志时使用的符号：

标志	标志=1	标志=0
OF	OV	NV
DF	DN	UP
IF	EI	DI
SF	NG	PL
ZF	ZR	NZ
AF	AC	NA
PF	PE	PO
CF	CY	NC

```
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0100  NU UP EI PL NZ NA PO NC
0B43:0100 F3 REPZ
0B43:0101 41 INC CX ←后面要执行的指令
```

4) T 命令（设置陷阱，单步执行）

① T：从当前 IP 开始执行一条指令。

```

-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0100 NU UP EI PL NZ NA PO NC
0B43:0100 F3 REPZ
0B43:0101 41 INC CX
-t
执行INC CX后
AX=0000 BX=0000 CX=0001 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0102 NU UP EI PL NZ NA PO NC
0B43:0102 42 INC DX
-t
执行INC DX后
AX=0000 BX=0000 CX=0001 DX=0001 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0103 NU UP EI PL NZ NA PO NC
0B43:0103 8D2B LEA BP,[BP+DI] SS:0000=20CD

```

② T 数值：从当前 IP 开始执行多条指令，数值:执行的指令条数。

② T=地址：从给定的地址执行一条指令

③ T=地址 数值：从给定的地址执行多条指令，数值:执行的指令条数。

-T；从当前 IP 开始执行一条指令

-T5；从当前 IP 开始执行 5 条指令

-T=100 5；从当前 0100H 开始执行 5 条指令

5) A 命令（将指令直接汇编成机器码输入到内存中。）

汇编命令 A

A [地址]：从指定的地址开始输入符号指令；如省略地址，则接着上一个 A 命令的最后一个单元开始；若第一次使用 A 命令省略地址，则从当前 CS:IP 开始（通常是 CS: 100）。

注释：

①在 DEBUG 下编写简单程序即使用 A 命令。

②每条指令后要按回车。

③不输入指令按回车，或按 Ctrl+C 结束汇编。

④支持所有 8086 符号硬指令，伪指令只支持 DB、DW，不支持各类符号名。

使用 A 命令在 0100H 开始输入指令 mov ax, 10 inc cx mov bl, al

```

-a 100
0B43:0100 mov ax, 10
0B43:0103 inc cx
0B43:0104 mov bl, al
0B43:0106

```

单步执行上述指令

```

-t=100
AX=0010 BX=0000 CX=0002 DX=0001 SP=FFEC BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0103 NU UP EI NG NZ NA PO NC
0B43:0103 41 INC CX
-t
AX=0010 BX=0000 CX=0003 DX=0001 SP=FFEC BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0104 NU UP EI PL NZ NA PE NC
0B43:0104 8BC3 MOV BL, AL
-t
AX=0010 BX=0010 CX=0003 DX=0001 SP=FFEC BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0106 NU UP EI PL NZ NA PE NC
0B43:0106 E8B4DF CALL EB0D

```

6) G 命令等的使用 (执行 A 命中输入的汇编指令)

运行程序命令 G

- ① G: 从 CS:IP 指向的指令开始执行程序, 直到程序结束或遇到 INT 3。
- ② G=地址: 从指定地址开始执行程序, 直到程序结束或遇到 INT 3。
- ③ G 断点 1[, 断点 2, ...断点 10]: 从 CS:IP 指向的指令开始执行程序, 直到遇到断点。
- ④G=地址 断点 1[, 断点 2, ...断点 10]

-G ; 从 CS:IP 指向的指令开始执行程序。

-G=100 ; 从指定地址开始执行程序。

-G=100 105 110 120

使用 A 命令在 0100H 开始输入指令 mov ax, 10 inc cx mov bl, al int 3

然后使用 g 命令执行

```
-a 100
0B43:0100 mov ax,10
0B43:0103 inc cx
0B43:0104 mov bl,al
0B43:0106 int 3
0B43:0107
-g=100
AX=0010 BX=0010 CX=0001 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0106 NU UP EI PL NZ NA PO NC
0B43:0106 CC INT 3
```

上面的例子设断点在 100H 处然后用 T 命令单步执行

```
-g=100 100
AX=0010 BX=0010 CX=0001 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0100 NU UP EI PL NZ NA PO NC
0B43:0100 B81000 MOV AX,0010
-t
AX=0010 BX=0010 CX=0001 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0103 NU UP EI PL NZ NA PO NC
0B43:0103 41 INC CX
-t
AX=0010 BX=0010 CX=0002 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0104 NU UP EI PL NZ NA PO NC
0B43:0104 88C3 MOV BL,AL
-t
AX=0010 BX=0010 CX=0002 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B43 ES=0B43 SS=0B43 CS=0B43 IP=0106 NU UP EI PL NZ NA PO NC
0B43:0106 CC INT 3
```

3. 用 DEBUG, 验证乘法、除法、加法、减法、带进位加、带借位减、堆栈操作指令、串操作指令的功能。

三. 实验要求

- 1. 仔细阅读有关 DEBUG 命令的内容, 对有关命令, 要求事先准备好使用的例子;

四. 实验环境

PC 微机

DOS 操作系统或 Windows 操作系统

MASM.EXE, LINK.EXE, DEBUG.COM 或宏汇编集成环境

五. 实验步骤

1. 在 DOS 提示符下, 进入 DEBUG 程序;
2. 在 DOS 目录下启动 DEBUG;
3. 详细记录每一步所用的命令, 以及查看结果的方法和具体结果。

六. 实验报告要求

1. 如何启动和退出 DEBUG 程序;
2. 整理每个 DEBUG 命令使用的方法, 实际示例及执行结果。

七. 思考题

启动 DEBUG 后, 要装入某一个.EXE 文件, 应通过什么方法实现?

实验六 内存操作数及寻址方法

一. 实验目的

- 1、熟练掌握 DEBUG 的常用命令，学会用 DEBUG 调试程序。
- 2、掌握数据在内存中的存放方式和内存操作数的几种寻址方式。
- 3、掌握简单指令的执行过程。

二. 实验内容

1、设堆栈指针 SP=2000H，AX=3000H，BX=5000H；请编一程序段将 AX 和 BX 的内容进行交换。请用堆栈作为两寄存器交换内容的中间存储单元，用 DEBUG 调试程序进行汇编与调试。

2、设 DS=当前段地址，BX=0300H，SI=0002H；请用 DEBUG 的命令将存储器偏移地址 300H~304H 连续单元顺序装入 0AH，0BH，0CH，0DH，0EH。

在 DEBUG 状态下送入下面程序，并用单步执行的方法，分析每条指令源地址的形成过程，当数据传送完毕时，AX 中的内容是什么。

程序清单如下：

```
MOV  AX, BX
MOV  AX, 0304H
MOV  AX, [0304H]
MOV  AX, [BX]
MOV  AX, 0001[BX]
MOV  AX, [BX][SI]
MOV  AX, 0001[BX][SI]
HLT
```

三. 实验要求

1、实验前要做好充分准备，包括汇编程序清单、调试步骤、调试方法，以及对程序结果的分析等。

2、本实验只要求在 DEBUG 调试程序状态下进行，包括汇编程序、调试程序和执行程序。

四. 实验报告

- 1、程序说明。说明程序的功能、结构。
- 2、调试说明。包括上机调试的情况、上机调试步骤、调试所遇到的问题是怎样解决的，并对调试过程中的问题进行分析，对执行结果进行分析。
- 3、写出源程序清单和执行结果。

实验七 汇编语言程序的编辑、编译、连接及调试方法

一、实验目的

学习和掌握汇编程序的编辑、编译、连接及调试方法

通过实验掌握下列知识: DOS 常用命令、EDIT、MASM、LINK、DEBUG 等命令的使用方法。

- 1) DOS 命令: DIR,CD
- 2) EDIT 编辑器
- 3) DEBUG 命令: A,D,E,G,R,T,U, Q。
- 4) 8086 寄存器: AX,BX,CX,DX,FLAGS,IP

二、实验内容

1) DOS 常用命令练习:

- ① 用 DIR 命令查看盘上文件 例: D:\DIR
- ② 用 CD 命令进入文件夹 例: D:\CD MASM
- ③ 用命令 COPY 复制一个文件。例: D:\COPY A.TXT E:\B.TXT

2) D:\MASM\

输入 EDIT HELLO.ASM

在程序编辑环境下输入程序如下:

DATA SEGMENT ;定义数据段

HEY DB 'HELLO,WORLD',0DH,0AH,'\$'

DATA ENDS

CODE SEGMENT ;定义代码段

ASSUME CS:CODE,DS:DATA

MAIN :

MOV AX,DATA

MOV DS,AX

MOV AH,9

MOV DX,OFFSET HEY

INT 21H

MOV AH,4CH ; 返回 DOS

INT 21H

CODE ENDS

END MAIN

编译: D:\MASM\MASM HELLO

编译无错时会产生一个 HELLO.OBJ 文件

连接： D:\MASM\LINK HELLO

连接无错时会产生一个 HELLO.EXE 文件

直接运行程序：

D:\MASM\HELLO.EXE

3) 根据参考程序，完成一个 $K=X+Y-Z$ (字运算) 的程序的编程及调试

参考程序：

两个数相加，结果放在 SUM 单元中， ($Z=X+Y$)

DATA SEGMENT ;定义数据段

NUM1 DW 2030H

NUM2 DW 4525H

SUM DW 00H

DATA ENDS

CODE SEGMENT ;定义代码段

ASSUME CS:CODE,DS:DATA

START: MOV AX,DATA

MOV DS,AX

MOV AX,[NUM1]

ADD AX,[NUM2]

MOV [SUM],AX

MOV AH,4CH ; 返回 DOS

INT 21H

CODE ENDS

END START

三. 实验要求

用汇编语言编写一个计算完成一个 $K=X+Y-Z$ (字运算)

四. 实验报告要求

写出在 DEBUG 状态下编写、运行程序的过程以及调试中所遇到的问题是如何解决的，并对调试过程中的问题进行分析，对执行结果进行分析。

实验八 屏幕字符显示程序

一、实验目的

通过实验掌握下列知识:

- 1、8086指令:XOR,INT等。
- 2、利用 DOS 功能调用 INT21H 的 2 号和 9 号功能进行屏幕显示的方法。

二、实验内容

(一)、利用INT 21 09H号功能调用显示字符串。

- 1、键入下列程序:

```
DATA SEGMENT
MSG DB 0DH,0AH, 'This is a sample!', 0DH,0AH,'$'
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE ,DS:DATA
START:  MOV AX,DATA
        MOV DS,AX
        LEA DX,MSG
        MOV  AH,9
        INT 21H
        MOV AH,4CH
        INT 21H
CODE ENDS
END START
```

编译并运行。

- 2、 利用INT 21H 2号功能显示字符

```
DATA SEGMENT
MSG DB 0DH,0AH, 'This is a sample!', 0DH,0AH,'$'
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE ,DS:DATA
START:  MOV AX,DATA
        MOV DS,AX
        XOR  DL,DL
        MOV BX, 0
        MOV CX,100
        LOP:MOV DL, MSG[BX]
        MOV  AH,2
```

```

        INT 21H
        PUSH CX
        MOV     CX,8000H
DELY:  PUSH CX
        MOV     CX,6000H
J:     LOOP     J
        POPCX
        LOOP    DELY
        POPCX
        INC BX
        LOOP    LOP
        MOV AH, 4CH
        INT 21H
CODE ENDS
EDN START

```

编译并运行。

编译，运行程序,观察屏幕上依次缓慢的显示的ASCII字符。仔细观察每个字符,是否和上个程序的结果一致。并分析从MOV CX，8000H 到LOOP DELY 程序段如果删除是否对程序显示结果有影响。

三．实验要求

- 1、试编写一个汇编语言程序，要求对键盘输入的小写字母用大写字母显示出来
- 2、编程提示：

利用DOS功能调用INT 21H的1号功能从键盘输入字符和2号功能在显示器上显示一个字符。

四、实验报告要求

观察并记录结果，回答提出的问题。

实验九 循环程序设计（该实验可选做）

一. 实验目的

掌握循环程序设计的方法

二、实验内容

1. 在内存单元 NUM 中定义了一个 16 位无符号数，统计其中为 1 的二进制位个数，结果存入内存字节单元 COUNT 中。

程序：

```
DATA SEGMENT
NUM DW 9A10H
COUNT DB ?
DATA ENDS
STK SEGMENT STACK
DW 50 DUP (?)
STK ENDS
CODE SEGMENT
ASSUME CS:CODE,DS:DATA,SS:STK
START:
MOV AX,DATA
MOV DS,AX
MOV AX,NUM ;把数移到 AX 中
AGAIN:
AND AX,AX ; 判断 AX 是否为 0，为 0 则退出
JZ EXIT
SHL AX,1
ADC COUNT,0
JMP AGAIN
EXIT: MOV AH,4CH
INT 21H
CODE ENDS
END START
```

用 DEBUG 命令调试程序，并观察实验结果即 COUNT 的值是否正确。

三. 实验要求

- 1、实验前要做好充分准备，包括汇编程序清单、调试步骤、调试方法，以及对程序结果的分析等。

2、 编写一个程序，要求比较两个字符串 STRING1 和 STRING2 所含字符是否相同，若相同则显示 ‘MATCH’ ， 若不相同则显示 ‘NO MATCH’ 。

3、 从键盘读入一个字符串(长度小于 80),统计字符”A”出现的次数

四、实验报告要求

- 1、程序说明。说明程序的功能、结构。
- 2、调试说明。包括上机调试的情况、上机调试步骤、调试所遇到的问题是怎样解决的，并对调试过程中的问题进行分析，对执行结果进行分析。
- 3、画出程序框图。
- 4、写出源程序清单和执行结果。